

Publishing an MQTT topic to Home Assistant from Atari BASIC via FujiNet

You can do quite a lot with FujiNet, and the N: device offers many of the possibilities.

I needed a little distraction, so I wanted to be able to set an MQTT topic from the Atari.

The N: device (<https://fujinetwork.github.io/fujinet-docs/features/network-device/>) supports many protocols, among them HTTP, HTTPS, FTP, SSH, ... and TCP.

If you ask Gemini, it claims that the N: device includes an MQTT protocol, which is simply not true. Gemini and I then settled on TCP and "recreated" MQTT on top of it a bit.

The test environment is a running Home Assistant (HAOS) with a Mosquitto broker running as an "app" (formerly add-on). It's also useful to have MQTT Explorer running, either as an app in Home Assistant or somewhere else on the network.

The basic principle is fairly simple:

Put the IP address of the Mosquitto broker into line 10.

The login "testuser" with the password "atari" is encoded as decimal byte values. Create this user either in Home Assistant or in the Mosquitto app. Alternatively, an existing user can be encoded into the BASIC listing instead (the length bytes then have to be adjusted as well, see below). After creating the user, check with MQTT Explorer that it can log in.

Step 1: connecting (MQTT CONNECT)

Lines 140 to 149 send the following individual bytes. All values are decimal. Length fields in MQTT are two bytes (high byte, low byte).

Position	Value (dec.)	Meaning
1	16	Packet type CONNECT: the client wants to connect
2	29	Remaining length: exactly 29 more bytes follow
3–4	0, 4	Length of the protocol name: 4 bytes follow
5–8	77, 81, 84, 84	"MQTT"
9	4	MQTT protocol level (4 = version 3.1.1)
10	194	Connect flags (binary 11000010): username and password follow, clean session
11–12	0, 60	Keep-alive: keep the connection open for 60 seconds
13–14	0, 0	Client ID length 0: the broker assigns an ID itself
15–16	0, 8	The username is 8 bytes long
17–24	116, 101, 115, 116, 117, 115, 101, 114	"testuser"
25–26	0, 5	The password is 5 bytes long
27–31	97, 116, 97, 114, 105	"atari"

Using a different user: besides the letters, change the two length fields (positions 15–16 and 25–26) and the remaining length in position 2. It is always $10 + 2 + 2 + \text{username length} + 2 + \text{password length}$; for testuser/atari: $10 + 2 + 2 + 8 + 2 + 5 = 29$.

Step 2: publishing the topic (MQTT PUBLISH)

I'd like to set the topic AtariXL on the broker. Later, this topic can be defined as a sensor in Home Assistant's configuration.yaml and used in automations, e.g. to switch some lamp on or off.

For this, the RETURN key works as a toggle switch: each press switches the value between "ON" and "OFF".

Of course you could also read the fire button of a joystick instead...

The payloads are in lines 190–193 and 220–223. Both are sent with the RETAIN flag set, so the broker keeps the last value and Home Assistant sees the current state even if it connects later.

Topic "AtariXL" with payload "OFF" (lines 190 to 193)

Position	Value (dec.)	Meaning
1	49	Packet type PUBLISH (48) + RETAIN flag (1), QoS 0: a topic is being sent
2	12	Remaining length: 12 more bytes follow
3-4	0, 7	The topic name is 7 characters long
5-11	65, 116, 97, 114, 105, 88, 76	"AtariXL"
12-14	79, 70, 70	Payload "OFF"

Topic "AtariXL" with payload "ON" (lines 220 to 223)

Position	Value (dec.)	Meaning
1	49	Packet type PUBLISH (48) + RETAIN flag (1), QoS 0: a topic is being sent
2	11	Remaining length: 11 more bytes follow
3-4	0, 7	The topic name is 7 characters long
5-11	65, 116, 97, 114, 105, 88, 76	"AtariXL"
12-13	79, 78	Payload "ON"

The complete program

Sorry, I haven't done anything in Atari BASIC for 30 years... The listing is unchanged from the original, including its German comments and screen texts (see the translations below it).

```
5 DIM IP$(60)
10 IP$="N:TCP://<IP-Adresse einsetzen>:1883/"
20 STATE=0
30 PRINT "MQTT-TCP BEREIT. RETURN DRUECKEN!"
40 REM *** TASTATUR-SPEICHER ABFRAGEN ***
50 TASTE=PEEK(764)
60 IF TASTE<>12 THEN GOTO 50
70 REM *** SPEICHER ZURUECKSETZEN ***
75 POKE 764,255
80 STATE=1-STATE:PRINT "SENDE STATE: ";
90 IF STATE=1 THEN PRINT "ON"
100 IF STATE=0 THEN PRINT "OFF"
110 REM *** VERBINDUNG OEFFNEN ***
120 OPEN #1,12,0,IP$
130 REM *** STEP 1: MQTT CONNECT (TESTUSER / ATARI) ***
140 PUT #1,16:PUT #1,29:PUT #1,0:PUT #1,4
141 PUT #1,77:PUT #1,81:PUT #1,84:PUT #1,84
142 PUT #1,4:PUT #1,194:PUT #1,0:PUT #1,60
143 PUT #1,0:PUT #1,0:PUT #1,0:PUT #1,8
```

```

146 PUT #1,116:PUT #1,101:PUT #1,115:PUT #1,116
147 PUT #1,117:PUT #1,115:PUT #1,101:PUT #1,114
148 PUT #1,0:PUT #1,5:PUT #1,97:PUT #1,116
149 PUT #1,97:PUT #1,114:PUT #1,105
150 REM *** STEP 2: MQTT PUBLISH SENDER ***
170 IF STATE=1 THEN GOTO 210
180 REM *** PAYLOAD OFF (RETAINED) ***
190 PUT #1,49:PUT #1,12:PUT #1,0:PUT #1,7
191 PUT #1,65:PUT #1,116:PUT #1,97:PUT #1,114
192 PUT #1,105:PUT #1,88:PUT #1,76
193 PUT #1,79:PUT #1,70:PUT #1,70
200 GOTO 230
210 REM *** PAYLOAD ON (RETAINED) ***
220 PUT #1,49:PUT #1,11:PUT #1,0:PUT #1,7
221 PUT #1,65:PUT #1,116:PUT #1,97:PUT #1,114
222 PUT #1,105:PUT #1,88:PUT #1,76
223 PUT #1,79:PUT #1,78
230 REM *** VERBINDUNG SCHLIESSEN ***
240 CLOSE #1
250 REM *** KLEINE PAUSE GEGEN DOPPEL-TRIGGERS ***
270 FOR I=1 TO 200:NEXT I
280 GOTO 40

```

Line	German text	English
10	<IP-Adresse einsetzen>	insert the broker's IP address here
30	MQTT-TCP BEREIT. RETURN DRUECKEN!	MQTT-TCP ready. Press RETURN!
40	TASTATUR-SPEICHER ABFRAGEN	poll the keyboard register (764; 12 = RETURN)
70	SPEICHER ZURUECKSETZEN	reset the register (255 = no key)
80	SENDE STATE:	sending state:
110	VERBINDUNG OEFFNEN	open the connection
230	VERBINDUNG SCHLIESSEN	close the connection
250	KLEINE PAUSE GEGEN DOPPEL-TRIGGERS	short pause against double triggers

Home Assistant

Home Assistant's configuration.yaml still needs to be extended:

```

mqtt:
  # ... your existing MQTT entries ...
  binary_sensor:
    - name: "Atari XL Button"
      state_topic: "AtariXL"

```

```
payload_on: "ON"  
payload_off: "OFF"  
device_class: "power"
```

After restarting Home Assistant Core, the new sensor is available and can be used in automations.

Notes and outlook

The Atari FujiNet approach can also serve as a template for other devices. I can well imagine that it would also work with the Schneider CPC and the M4; maybe I'll try that in the next few days. For the C64, Meatloaf is also a good option.

In principle, it works with anything that can send individual bytes over TCP.

Everything can still be improved. For example, the Atari doesn't yet notice the acknowledgement (CONNACK) that Mosquitto sends back, and it closes the connection without an MQTT DISCONNECT, which the broker tolerates. I built it on a notebook with a HAOS VM, the Atari XL emulator Altirra (<https://virtualdub.org/altirra.html>) and a FujiNet emulator in the form of a matching bridge from the FujiNet project (<https://github.com/FujiNetWiFi/fujinet-emulator-bridge>). The real test was done with an unmodified Atari XL + FujiNet. Our Home Assistant has been running since 2021...

What can you do with it now? Good question... Build an 8-bit control panel for Home Assistant for the retro computer room? A back channel would be nice too, to show the status on the Atari XL.

Created with the help of AI (Gemini). The guide was mostly written "by hand" and may therefore contain errors 😊...

Contact: joergr via the forum "<https://forum.classic-computing.de/forum/>" of the VzEKC e.V. (Verein zum Erhalt klassischer Computer e.V., the association for the preservation of classic computers).

Corrections compared to the German original

- ON table: position 2 means "11 bytes follow" (the original said 12). The program was already correct.
- ON table heading: "Topic 'AtariXL' with payload 'ON'" (the original said "Payload 'AtariXL'").
- Two-byte length fields are now shown as pairs, and the meaning of 16 (CONNECT), 49 (PUBLISH + RETAIN) and 194 (username + password + clean session) is spelled out.
- The byte sequences of the listing were checked on 2026-09-27 against a local Mosquitto 2 broker: the connection was accepted, and both ON and OFF were stored as retained values and delivered to a later subscriber.